

Лабораторна робота №14

14.1.1. Для чого застосовується шаблон проєктування Ланцюжок обов'язків?

Ланцюжок обов'язків — це поведінковий шаблон проєктування, який дозволяє передавати запити послідовно через ланцюжок обробників. При надходженні запиту кожен обробник вирішує, чи може він його опрацювати, чи передати далі наступному обробнику в ланцюжку.

Основна мета застосування:

- **Декаплінг (розв'язання залежностей):** Послаблює зв'язок між відправником запиту та отримувачем, оскільки відправник не знає, хто саме з ланцюжка обробить запит.
- **Динамічне призначення обов'язків:** Дозволяє змінювати склад та порядок обробників під час виконання програми.
- **Реалізація принципу єдиної відповідальності (SRP):** Кожен клас-обробник відповідає лише за певний тип логіки або умови обробки.
- **Гнучкість:** Замість одного масивного умовного оператора (`if-else` або `switch`), логіка розподіляється між незалежними класами.

Приклади використання: системи логування, обробка подій інтерфейсу, перевірка прав доступу, фільтрація HTTP-запитів.

14.1.2. Наведіть основні етапи реалізації шаблону Ланцюжок обов'язків.

Для правильної реалізації шаблону необхідно виконати наступні кроки:

1. **Створення інтерфейсу або абстрактного класу обробника:**

- Визначити метод для обробки запиту (наприклад, `handle()`).
- (Опціонально) Визначити метод для встановлення наступного обробника в ланцюжку (наприклад, `setNext()`).

2. Реалізація базового класу обробника (опціонально, але рекомендовано):

- Створити поле для збереження посилання на наступний обробник.
- Реалізувати стандартну поведінку передачі запиту: якщо обробник не може виконати запит, він викликає метод обробки у наступного об'єкта.

3. Створення конкретних обробників:

- Успадкувати класи від базового обробника.
- Кожен конкретний клас має реалізувати логіку: «Чи можу я обробити цей запит?». Якщо так — обробляє, якщо ні — делегує виконання суперкласу або безпосередньо наступному ланці.

4. Формування ланцюжка (на стороні клієнта):

- Клієнт створює екземпляри конкретних обробників.
- Клієнт з'єднує їх у потрібній послідовності за допомогою методів `setNext()` або через конструктори.

5. Запуск запиту:

- Клієнт передає запит першому обробнику в ланцюжку. Запит автоматично проходить через усі ланки, поки не буде знайдено відповідного виконавця (або поки ланцюжок не закінчиться).